

FIKRATAK DIGITAL · فكرتك الرقمية

ENGINEERING IMPLEMENTATION BRIEF

Google OAuth Authentication

+ Role-Based Access Control (RBAC)

مصادقة جوجل + نظام صلاحيات متعدد المستويات

TARGET DEPLOYMENTS

nazrah-client.algarni.com

nazrah-app.algarni.online

Prepared for · Mohammed Abotargah

Project Manager & Lead Engineer · Technical Entrepreneurship 2026

01 · Executive Summary

This brief delivers a production-ready plan to add Google OAuth authentication and a three-tier Role-Based Access Control (RBAC) system to both Nazrah deployments. The package contains three artifacts: (1) the architectural plan, (2) a single copy-paste prompt for Claude Code, and (3) the rationale explaining each decision.

هذا الموجز يقدم خطة جاهزة للإنتاج لإضافة مصادقة جوجل ونظام صلاحيات بثلاث طبقات لكلا نشرتي تطبيق نظرة. الحزمة تحوي ثلاثة مكونات: الخطة المعمارية، برومبت واحد جاهز للصق في Claude Code، والمبررات التي تشرح كل قرار.

Scope at a glance

- ◆ Google Sign-In enabled on both subdomains via a single OAuth Client with multiple authorized origins.
- ◆ Three role tiers: normal_client, premium_client, internal_management — enforced server-side.
- ◆ Bootstrap admin: tarj123@gmail.com auto-promoted to internal_management on first login; password set for email/password fallback.
- ◆ Audit log for every role change. No client-side role checks for sensitive actions.

⚠ SECURITY ADVISORY · تنبيه أمني

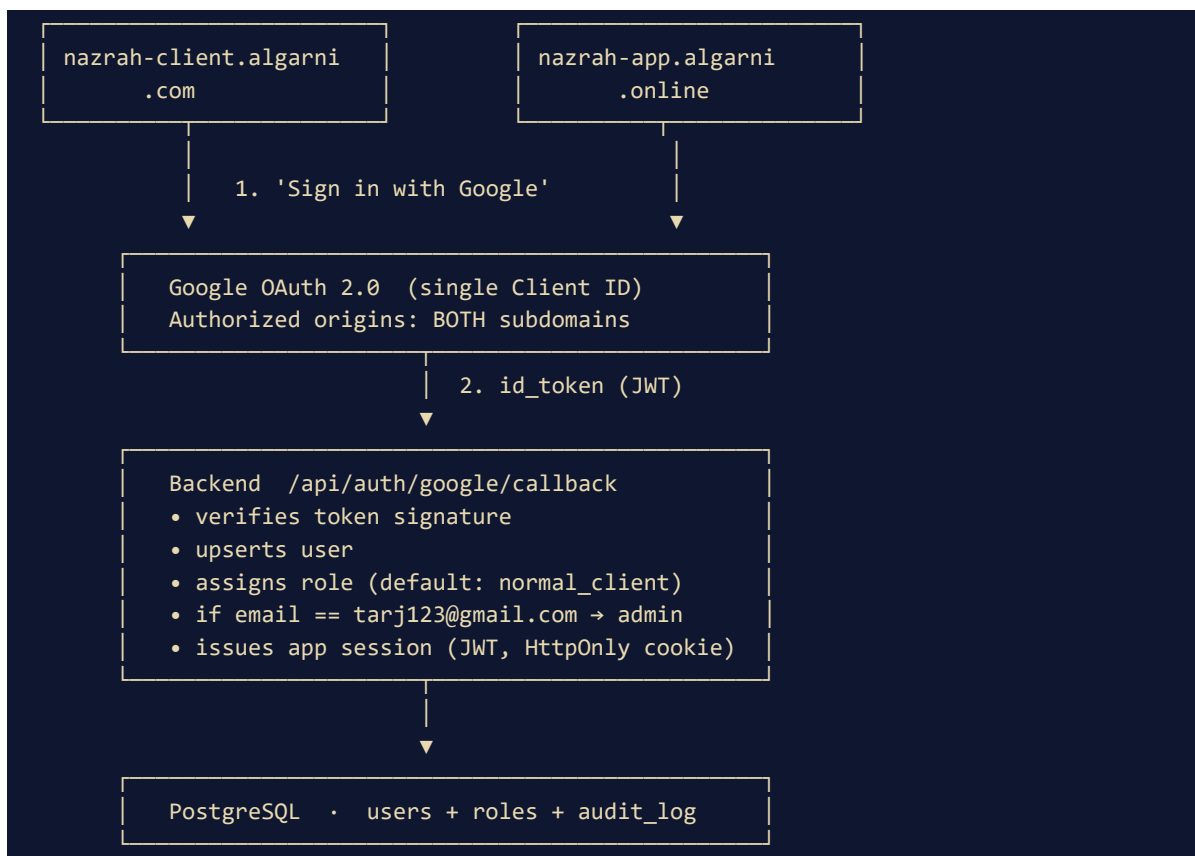
The password Murad@225144 was shared in plaintext during this request. Before deploying, please: (1) change this password immediately after first successful login; (2) enable 2FA on tarj123@gmail.com; (3) store the credential only in a password manager — never in code, git, or chat history.

كلمة المرور أُرسِلت بنص واضح في الطلب. يُرجى تغييرها فور أول تسجيل دخول، وتفعيل المصادقة الثنائية على البريد، وعدم تخزينها في الكود أو git أو سجل المحادثات.

02 · Architecture Overview

Both subdomains are treated as separate frontends that share a single identity & authorization backend. This avoids duplicate user records, lets one admin govern both, and keeps Google's OAuth quota usage minimal.

Data flow



Role matrix

Role	Capabilities	Default
normal_client	Read public content · save preferences · basic dashboard · standard limits	Yes (default)
premium_client	Everything above + premium content · higher rate limits · priority support flag · advanced exports	No
internal_management	Full admin · promote/demote users · view audit log · manage content · system settings	No (admin)

03 · Implementation Plan (Phased)

Phase 1 · Google Cloud Console Setup

- ◆ Open console.cloud.google.com → create (or reuse) project 'Nazrah Production'.
- ◆ APIs & Services → OAuth consent screen → External → fill app name, support email, logo, privacy URL.
- ◆ Add scopes: openid, email, profile.
- ◆ Add test users while in 'Testing' state (include tarj123@gmail.com).
- ◆ Credentials → Create Credentials → OAuth Client ID → Web application.
- ◆ Authorized JavaScript origins: https://nazrah-client.algarni.com and https://nazrah-app.algarni.online
- ◆ Authorized redirect URIs: /api/auth/google/callback on BOTH subdomains.
- ◆ Save the Client ID and Client Secret — they will go into environment variables, never into git.

Phase 2 · Database Migration

Add the role column, audit table, and seed the admin record. Run this once on the shared database.

```
-- 001_add_rbac.sql
ALTER TABLE users
  ADD COLUMN IF NOT EXISTS role TEXT NOT NULL DEFAULT 'normal_client'
  CHECK (role IN ('normal_client','premium_client','internal_management')),
  ADD COLUMN IF NOT EXISTS google_sub TEXT UNIQUE,
  ADD COLUMN IF NOT EXISTS provider TEXT DEFAULT 'email',
  ADD COLUMN IF NOT EXISTS last_login_at TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS promoted_at TIMESTAMPTZ;

CREATE TABLE IF NOT EXISTS role_audit (
  id BIGSERIAL PRIMARY KEY,
  target_user_id BIGINT NOT NULL REFERENCES users(id),
  changed_by BIGINT REFERENCES users(id),
  old_role TEXT,
  new_role TEXT NOT NULL,
  reason TEXT,
```

```

    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_users_role ON users(role);
CREATE INDEX IF NOT EXISTS idx_users_email ON users(LOWER(email));

```

Phase 3 · Backend Auth Routes

- ◆ POST /api/auth/google → receive id_token from frontend, verify with google-auth-library, upsert user.
- ◆ On upsert: if email == tarj123@gmail.com AND role != internal_management → promote and write to role_audit.
- ◆ POST /api/auth/login → email/password fallback (for tarj123 with Murad@225144 — hashed via bcrypt, cost 12).
- ◆ POST /api/auth/logout → clear HttpOnly cookie.
- ◆ GET /api/me → return { id, email, role, name, picture }.
- ◆ PATCH /api/admin/users/:id/role → internal_management only; writes audit row.

Phase 4 · Frontend Integration

- ◆ Install @react-oauth/google (or equivalent for the stack on each app).
- ◆ Wrap root in <GoogleOAuthProvider clientId={...}> using VITE_GOOGLE_CLIENT_ID / NEXT_PUBLIC_GOOGLE_CLIENT_ID.
- ◆ Render <GoogleLogin onSuccess={...}> on the login page — RTL-aware styling, dark premium variant.
- ◆ On success → POST id_token to /api/auth/google → receive session cookie → redirect by role.
- ◆ Add a useAuth() hook that exposes { user, role, isAdmin, isPremium, login, logout }.
- ◆ Render role-gated UI: PremiumGate, AdminPanel, NormalDashboard — but ALWAYS re-check role on the server.

Phase 5 · Admin Panel

- ◆ Route /admin/users — table of users with current role and last login.
- ◆ Dropdown per row: normal_client | premium_client | internal_management.
- ◆ PATCH /api/admin/users/:id/role with optional reason field → toast confirmation.
- ◆ Route /admin/audit — paginated view of role_audit, filterable by user.

Phase 6 · Verification

- ◆ Smoke test on both subdomains: Google login → /api/me returns the correct role.
- ◆ Log in as tarj123@gmail.com → confirm role == internal_management on first login.
- ◆ Log in as a new account → confirm role == normal_client.
- ◆ Promote a test user from the admin panel → confirm audit row appears.

- ◆ Attempt PATCH /api/admin/users/... as a normal_client → must return 403.
- ◆ Run Lighthouse + browser console scan — no client secrets, no client_id in non-public bundles.

04 · Environment Variables

Add these to .env on each deployment. Never commit them. Use the Hostinger / VPS secret manager.

```
# Shared (both apps)
GOOGLE_CLIENT_ID=xxxxxxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxx
JWT_SECRET=<openssl rand -hex 64>
SESSION_COOKIE_NAME=nazrah_sid
SESSION_COOKIE_DOMAIN=.algarni.com # adjust per app
BOOTSTRAP_ADMIN_EMAIL=tarj123@gmail.com

# Frontend (PUBLIC – these are safe to ship to browser)
VITE_GOOGLE_CLIENT_ID=xxxxxxxxxxx.apps.googleusercontent.com
# or for Next.js:
NEXT_PUBLIC_GOOGLE_CLIENT_ID=xxxxxxxxxxx.apps.googleusercontent.com
```

05 · The Prompt · Copy-Paste into Claude Code

Paste the entire block below in one go. Claude Code will read your project structure first, then execute the phases. Replace the two placeholders before running.

■ PROMPT · CLAUDE CODE

ROLE:

You are a senior full-stack engineer. Implement Google OAuth and a 3-tier RBAC system across BOTH apps in this workspace:

- nazrah-client.algarni.com (client-facing app)
- nazrah-app.algarni.online (app users)

Both share a single backend and a single PostgreSQL database.

STEP 0 · DISCOVERY

Before writing code, scan both projects and report:

1. The framework of each frontend (Next.js / Vite-React / other).
2. The backend stack (Express / Fastify / Next API routes / NestJS).
3. The existing auth implementation if any.
4. The ORM and migration tool (Prisma / Drizzle / raw SQL / Knex).
5. The current users table schema.

Stop and ask me to confirm before continuing if anything is ambiguous.

STEP 1 · GOOGLE OAUTH

- Install google-auth-library on the backend.
- Install @react-oauth/google (or framework-appropriate equivalent) on BOTH frontends.
- Wrap the app root in GoogleAuthProvider using the env var NEXT_PUBLIC_GOOGLE_CLIENT_ID or VITE_GOOGLE_CLIENT_ID.
- Add a 'Sign in with Google' button to the login page of each app, styled to match the dark premium aesthetic and RTL-aware when the locale is Arabic.
- Create POST /api/auth/google that accepts an id_token, verifies it against Google, then upserts the user.
- Set an HttpOnly, Secure, SameSite=Lax session cookie. Cookie domain must be configured per app (.algarni.com / .algarni.online).

STEP 2 · RBAC SCHEMA

Generate the migration shown in the brief:

- Add columns: role, google_sub, provider, last_login_at, promoted_at.
- Enforce CHECK constraint: role IN ('normal_client', 'premium_client', 'internal_management').
- Default role: 'normal_client'.
- Create the role_audit table.
- Add indexes on role and LOWER(email).

Use the project's existing migration tool. Do NOT bypass migrations.

STEP 3 · BOOTSTRAP ADMIN

- Read BOOTSTRAP_ADMIN_EMAIL from env (= tarj123@gmail.com).
- On every successful login, if the user's email equals this value AND role != 'internal_management', promote them and write a row to role_audit with reason = 'bootstrap admin'.
- Also create a seed/admin script that lets me set an email+password fallback for this user. Hash the password with bcrypt (cost 12). Read the password from a CLI prompt or an env var ADMIN_BOOTSTRAP_PW – never hard-code it in source.

STEP 4 · ADMIN ENDPOINTS

- GET /api/me → { id, email, role, name, picture }
- PATCH /api/admin/users/:id/role
body: { role, reason? }
requires: caller.role === 'internal_management'
writes role_audit row
- GET /api/admin/users (paginated list)
- GET /api/admin/audit (paginated audit log)

All admin endpoints MUST verify the role server-side from the session, never trust a value sent by the client.

STEP 5 · FRONTEND

On each app:

- Add a useAuth() hook exposing { user, role, isAdmin, isPremium, login, logout }.
- Build a /admin/users page (internal_management only) with a role dropdown per user and an optional 'reason' field on change.
- Build /admin/audit showing the role_audit log.
- Show role-appropriate UI on the dashboard (PremiumGate, etc.).

- Server must re-check role on every protected request — frontend gating is for UX only.

STEP 6 · ENV & SECRETS

Add to `.env.example` (NOT `.env`): `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `JWT_SECRET`, `SESSION_COOKIE_NAME`, `SESSION_COOKIE_DOMAIN`, `BOOTSTRAP_ADMIN_EMAIL`, and the public client id var. Update `.gitignore`. Print a clear checklist of values I must fill into the real `.env` on the Hostinger VPS.

STEP 7 · TESTS

- Unit: `verifyGoogleToken`, `promoteToAdmin`, `requireRole` middleware.
- Integration: full login flow + role enforcement (403 for unauthorized PATCH).
- Add a README section 'Auth & Roles' explaining the model.

STEP 8 · DEPLOY CHECKLIST

Print, at the end:

- Google Cloud Console steps (authorized origins + redirect URIs).
- Migration command for production.
- Command to bootstrap the admin password.
- Smoke-test script for both subdomains.

CONSTRAINTS

- Do NOT log `id_token`, `password`, or `JWT_SECRET` — anywhere.
- Do NOT commit `.env`. Verify `.gitignore` covers it.
- Default to least privilege; new users are always `normal_client`.
- Arabic RTL must work for the login button label.
- Code style follows each project's existing conventions.

Begin with STEP 0 and wait for my confirmation.

06 · Why This Plan · Design Rationale

Why one OAuth Client for both subdomains?

Google allows multiple authorized JavaScript origins per client. Sharing a client means one consent screen, one set of secrets to rotate, and unified analytics in Google Cloud. The cost: a misconfiguration affects both apps — mitigated by adding both origins in a single change and smoke-testing immediately.

Why server-side role checks only?

Frontend role checks are cosmetic; a determined user can edit React state or call the API directly. The `PATCH /api/admin/users/:id/role` endpoint will refuse any caller whose session does not resolve to `internal_management` — even if the UI shows the button. This is the only model that survives a hostile client.

Why the bootstrap-on-login pattern for tarj123@gmail.com?

Hard-coding an admin user_id in SQL is brittle: a fresh migration on a new database leaves no admin. Promoting on first login (when the email matches the env-configured BOOTSTRAP_ADMIN_EMAIL) is idempotent, transparent in the audit log, and survives database resets. The email is the source of truth.

Why bcrypt for the fallback password?

Google OAuth is primary; the email/password path exists only as a recovery channel if Google ever blocks the account. bcrypt at cost 12 is the current OWASP recommendation — slow enough to resist offline cracking, fast enough for interactive login. The password lives only as a hash in the database.

Why a separate role_audit table?

Compliance and forensics. If a user suddenly has internal_management access, you must be able to answer 'who promoted them, when, and why' within seconds. A dedicated table — append-only, never updated — is the simplest defensible record.

Why HttpOnly, SameSite=Lax cookies and not localStorage?

localStorage is readable by any JavaScript on the page, which means a single XSS vulnerability hands attackers a session token. HttpOnly cookies are unreachable from JS. SameSite=Lax prevents CSRF for state-changing requests while still allowing top-level navigation from external links.

Why phased rollout instead of one giant PR?

Each phase has an independent verification step. If Phase 2 (migrations) fails on production, the app keeps running on the existing auth. If Phase 4 (frontend) has a styling regression, the backend already works and can be tested via curl. Small, reversible steps are the engineer's insurance policy.

07 · Estimated Timeline

Phase	Effort (focused)	Dependencies
1 · Google Console	20 – 40 min	Owner Google account access
2 · Migration	30 min	DB credentials
3 · Backend routes	2 – 3 h	Phase 2 complete
4 · Frontend (both apps)	2 – 3 h	Phases 1 & 3 complete
5 · Admin panel	2 h	Phase 4 complete
6 · Verification & deploy	1 h	All previous phases

Total focused effort: roughly one full working day, or two half-days split between backend and frontend. Add buffer for the Google verification screen if you intend to publish the consent screen out of 'Testing' mode.

ملخص بالعربية · 08

هذا المستند يقدم خطة هندسية متكاملة لإضافة تسجيل الدخول عبر جوجل ونظام صلاحيات بثلاث طبقات لكلا تطبيقَي نظرة. الطبقات هي: عميل عادي، عميل مميز، وإدارة داخلية. سيتم تعيين البريد tarj123@gmail.com تلقائيًا كمسؤول كامل الصلاحيات عند أول تسجيل دخول.

المراحل الست:

- ◆ إعداد مشروع Google Cloud وتفعيل OAuth مع إضافة كلا النطاقين في الـ Origins المسموحة.
- ◆ ترقية قاعدة البيانات بإضافة عمود role وجدول role_audit.
- ◆ بناء مسارات المصادقة في الـ backend مع التحقق من id_token جانب الخادم.
- ◆ ربط الواجهتين الأماميتين بزر تسجيل دخول بجوجل بتصميم داكن فاخر يدعم العربية.
- ◆ بناء لوحة الإدارة لتغيير الصلاحيات مع سجل تدقيق كامل.
- ◆ اختبار شامل على النطاقين قبل الإنتاج.

الأمان أولًا: لا تُحفظ كلمة المرور في الكود، ويتم التحقق من الصلاحية في الـ backend دائمًا، ولا يُعتمد أبدًا على التحقق من جهة المتصفح للعمليات الحساسة.

09 · Next Action

1. Replace BOTH placeholders in the prompt (Client ID, Client Secret will be added by Claude Code via .env).
2. Open Claude Code in your workspace that contains both Nazrah projects.
3. Paste the entire prompt from Section 05.
4. Confirm the discovery report it returns before authorizing Step 1.
5. After Phase 6 verification, change the bootstrap password and enable 2FA on tarj123@gmail.com.

— End of Brief —